COFORGE FDE DELIVERY · ASSOCIATE FDE CAPSTONE, LUMEN BANK DISPUTE COPILOT

AWS Pilot Deployment: Executive Brief

Card Dispute Intake and Evidence Copilot. One container, one load balancer, one audited human gate: architecture, controls, cost, and the record of the 28 October deployment.

DOCUMENT ID
CFG-FDE-AWS-001

VERSION
2.1

STATUS
Issued, pilot deployed in us-east-1

DATE
28 October 2026

OWNER
Coforge FDE Pod

AUDIENCE
Coforge delivery leadership; Lumen Bank platform, operations and model risk reviewers

CLASSIFICATION
Internal

Fictional training material. All institutions, people, policies, records and currency in this document are fictional. The pilot described here was deployed on 28 October 2026 to AWS account 104005345381 in us-east-1 and exercised end to end in the cloud. Every figure marked cloud is a measurement from that deployment, not a forecast.

Internal · Fictional training data

Contents

1 Executive summary 3
1.1 What the pilot is 3
1.2 What the move to AWS changes 4
1.3 The ask 4
2 Why this, why now 4
3 What is being deployed 5
3.1 The four surfaces 5
3.2 What is deliberately absent, and why 5
3.3 One process plays two roles, and that is the gap a reviewer will find 6
4 Architecture as deployed 6
4.1 Request flow 7
4.2 Component choices and the constraint that forced each 8
4.3 Options considered and rejected 8
4.4 Status, stated plainly 8
5 Controls, and what a bank reviewer will still reject 9
5.1 What is true of this deployment 9
5.2 What a reviewer would still object to, with the answer for each 9
5.3 What this document is not 10
6 Cost 10
6.1 Price basis 10
6.2 Scenarios 11
6.3 Guardrails 11
6.4 What is deliberately not built, with the monthly cost avoided 11
7 Delivery plan 12
7.1 Effort, by stage 12
7.2 Prerequisites 12
7.3 Verification before anyone watches 12
7.4 Who does what 13
8 Operating the pilot 13
8.1 Seven beats, each one a claim on screen 13
8.2 Fallback ladder 14
8.3 Monitoring during the window 14
8.4 Failure modes and the fix 14
8.5 Teardown, in order, with confirmation 14
9 Risk register 15
9.1 The two risks that actually decide whether this works 15
10 From pilot to production 16
10.1 The three shortcuts, each with a named replacement 16
10.2 Phases and entry criteria 16
10.3 If the model is ever turned on, it is Bedrock 16
11 Decisions and status 17
12 Assumptions and limits 17
Appendix A, verification evidence, 28 October 2026 18
Appendix B, command reference 19
Appendix C, source register 20

1 Executive summary

In one line: the copilot left the laptop. On 28 October 2026 it was deployed to AWS as one container behind one load balancer, answered on a public amazonaws.com hostname, scored 12 of 12 in the cloud, and ran all seven demo beats against real cloud state, including a human approval issued through ECS Exec and a duplicate write refused with 409. It costs about $0.45 for a demo day, stands up in about 95 minutes, and tears down in 3. The four decisions in section 11 now concern what happens after the pilot window.

1.1 What the pilot is

The card dispute copilot reads a bank complaint, collects the linked records, and prepares one cited working record for a human operator. It is a Python service with a contract engine, an evaluation suite, a human review gate, and an audit trail. It has no user interface, no live model calls in the default path, and no database server.

The pilot takes the same container image and puts it behind a public Application Load Balancer on ECS Fargate in us-east-1. One task, one state file, two least-privilege IAM roles, logs in CloudWatch, and an operator approval channel that runs inside the cloud task rather than on a laptop.

Measure	Value	Basis
Dev cases passed, cloud	12 of 12	scored run executed inside the Fargate task through ECS Exec, 28 Oct 2026
Draft latency, p50 and p95, cloud	17 ms and 3.96 s	same run, nearest-rank over 12 cases. The tail is retry backoff on CARD-D07 and CARD-D10
Cost per case	$0.00	0 model calls, 0 tokens in the meter file
Source under test	5,472 lines across 21 modules	src/ count at commit c7ea9b2
Automated checks	5 CI jobs: lint, bandit, pytest, pytest-windows, env-files	.github/workflows/checks.yml, 100 percent coverage gate
Container image	342 MB, deployed	built from deploy/poc/Dockerfile at commit c7ea9b2, pushed to ECR as ccd-poc-api:poc
Infrastructure as code	16 Terraform files, applied	26 resources in account 104005345381, terraform 1.17.0 with AWS provider 6.70
Live endpoint	http://ccd-poc-alb-1873782563.us-east-1.elb.amazonaws.com	ALB DNS, us-east-1, reachable only from the operator address
Pilot run-rate	$0.056 per hour, $1.35 per day	AWS Price List API, us-east-1, 28 Oct 2026
Demo day	about $0.45 for 8 hours	same
Stand-up effort	95 minutes, first attempt, including three script defects	measured, section 7
Teardown	about 3 minutes, billing stops	one command, section 8.5
AWS spend, deployed	about $0.04 at the time of writing	40 minutes of one task and one ALB at the published rates

Revision 2.1, 28 October 2026. Section 3.3 was added after a design review asked whether the SQLite file in the container is the customer database. It is not: the five tables are the platform's workflow state, the customer data is the fictional bank's dataset served read-only over HTTP, and the two share a process. Section 3.3 states that custody gap plainly, and sections 5.2, 9 and 10.1 now answer it by name. No code changed for this revision.

Revision 2.0, 28 October 2026. Version 1.2 of this brief described a target state. That target state now exists. The stack was applied in six phases to account 104005345381, the image was pushed to ECR, the service came up

healthy behind the ALB, and the seven beats ran against the deployed task. Appendix A carries the cloud outputs, section 4.4 states what is and is not proven, and three defects found during the deployment are recorded rather than smoothed over: the phase script did not target the VPC network resources, the image build used the wrong context, and the ECS Exec logging call named an API parameter that does not exist in this version.

Revision 1.1, 28 October 2026. Version 1.0 of this brief carried the 27 October measurement: 9 of 12 cases, p95 10.8 ms, with CARD-D07, CARD-D09 and CARD-D10 failing. The D09 statement fix and the retry handling for the two bank-read cases landed on main that evening. Every measured figure below was re-read or re-executed for this revision, the KPI table from the published run at commit 8c3ac7f and Appendix A from a container built at c7ea9b2.

1.2 What the move to AWS changes

It changes	It does not change
The service boundary becomes public and real, not a localhost port	The score: 12 of 12 cases pass in the cloud because they pass in the container
The human approval becomes a cloud-side state change, logged in CloudWatch	The engine, the contracts, the dataset, the rules
The audit trail lives where a bank reviewer can reach it	Cost per case, which stays at zero model calls
An IAM boundary exists between the service and the account	The refusal path: unapproved writes still return 403
"Where does it run" gets an artifact instead of an assurance	Data handling: the dataset remains fictional training data

The honest framing to repeat to any audience: **deployment does not launder correctness.** The value of the pilot is that it moves the boundary, the human gate and the audit trail into the cloud, and that it can be stood up and destroyed on demand for the price of a coffee.

1.3 The ask

Four decisions, detailed in section 11: approve a capped spend envelope on one sandbox account; choose the transport security stance for demo day; sanction the "same score, different boundary" framing for the panel; and set a date for the production hardening review. The first is the only one that costs money, and the cap proposed is $25.

2 Why this, why now

The capstone is in its final leg and its evidence today is a laptop. Three audiences ask three different questions about the same system.

Audience	Their question	What answers it
Capstone panel	Can this team ship, and can the work be defended?	Score, contracts, audit trail, demo
Coforge delivery leadership	Is there a credible transition story beyond the capstone?	A priced, controlled, reversible pilot
Lumen Bank platform and risk	Would this survive our review, and what would it cost to make it?	Controls matrix, named gaps, production path

A pilot answers the second question properly and the third partially: it demonstrates the boundary and the human gate in a real cloud account, and it lists, without softening, what a bank audit would still reject. It does not pretend to answer the security questionnaire.

Two constraints shape the design. First, the system's deployable surface is genuinely small: a read-only bank API, a review queue, and an audit table. Second, the capstone's own gate is a 70 out of 100 rubric recorded in docs/POD_PLAN.md, and that rubric contains no cloud criterion. So this is not gate-driven work. It is a deliberate, low-cost investment in the transition story, and the cost of being wrong is 45 cents.

3 What is being deployed

3.1 The four surfaces

Everything the pilot exposes is one of four things: reading bank records, requesting a review, creating a record after an approved review, and reading the audit trail. The service is deliberately narrow.

Surface	Endpoints	Who calls it
Health and inventory	GET /health, GET /v1/cases	Operator, load balancer health check
Case evidence, read only	GET /v1/cases/{case}/{tool} (transactions, merchant)	The draft builder
Review queue	POST /v1/reviews, GET /v1/reviews/{id}	Operator, demo script
Record creation and readback	POST /v1/records (Idempotency-Key), GET /v1/records/{case}	Operator after approval

KYC writes are refused in code, not by network policy: every creation path returns 403 read_only_mission. Approval is bound to a SHA-256 hash of the draft, so a draft edited after approval is refused with 403 approval_missing_or_draft_changed. Duplicate logical keys return 409 with the existing record id.

3.2 What is deliberately absent, and why

Absent	Why, for this pilot	When it arrives
Live model calls	models.toml registers the model with an empty approved_by, and the client refuses a live pair without a passed, bank-approved entry. The scored run makes 0 calls.	Phase 2, behind a qualification run and a signature
Web interface	The operator path is a CLI by design; adding a browser console adds an admin surface the model could reach	Phase 2, with authentication
User authentication	There are no users. The write path is gated by an approved review, not by identity	Phase 1, Cognito or an IAM authorizer
Database service	SQLite is single-writer and in-process. RDS or Aurora would force a persistence adapter, which is a code change, not a deployment change	Phase 1, behind the existing store seam
TLS on the demo endpoint	ACM needs a domain you control. The demo runs on a one-address allowlist over HTTP with fictional data	Phase 1, or CloudFront from day one if a bank stakeholder joins

3.3 One process plays two roles, and that is the gap a reviewer will find

The deployed image serves two things that a bank keeps apart. In the same Python process, on the same disk, under one task role:

- **The fictional bank.** GET /v1/cases and GET /v1/cases/{case}/{tool}, read-only, from the dataset loaded at startup.
- **The dispute platform.** The review queue, record creation, idempotency ledger and audit trail, which are the write path.

  It is worth being precise about what lives where, because the two are easy to conflate.

Thing	Where it actually is	Whose data it is
Card, transaction and merchant records	data/dev/dataset.json, loaded at start (tools/runtime.py:48) and served over HTTP only	The bank's
Reviews, records, idempotency keys, attempt counters, audit	SQLite, five tables (tools/runtime.py:29), never read from disk by the copilot	The platform's
The dataset in the image	Copied in at build time (deploy/poc/Dockerfile:32)	The bank's, held in the wrong container

So the honest answer to "does the write path hold the customer database" is yes: same process, same ephemeral disk, same IAM role, with fictional data. The read-only guarantee is a code rule (403 read_only_mission), not custody.

It stays that way for the pilot on purpose. Splitting the two services means a scope switch inside create_server, two base URLs in the CLI clients, and tests for both, under a 100 percent coverage gate, on the exact path that produces the 12 of 12 run. That is half a day of code the night before a defence, in exchange for a boundary that a reviewer would still qualify, because two containers in one task share an ENI, a disk and a role. Section 10.1 names the replacement: a separate bank service on its own read-only contract, with no customer data in the copilot image at all. Say this before the reviewer asks, not after.

4 Architecture as deployed

The design is one container, one load balancer, two IAM roles and one state file. The account had no default VPC, so the stack creates a dedicated VPC at 10.42.0.0/16 with two public subnets and an internet gateway, and it deliberately does not create a NAT gateway, a database, an S3 bucket or a domain.

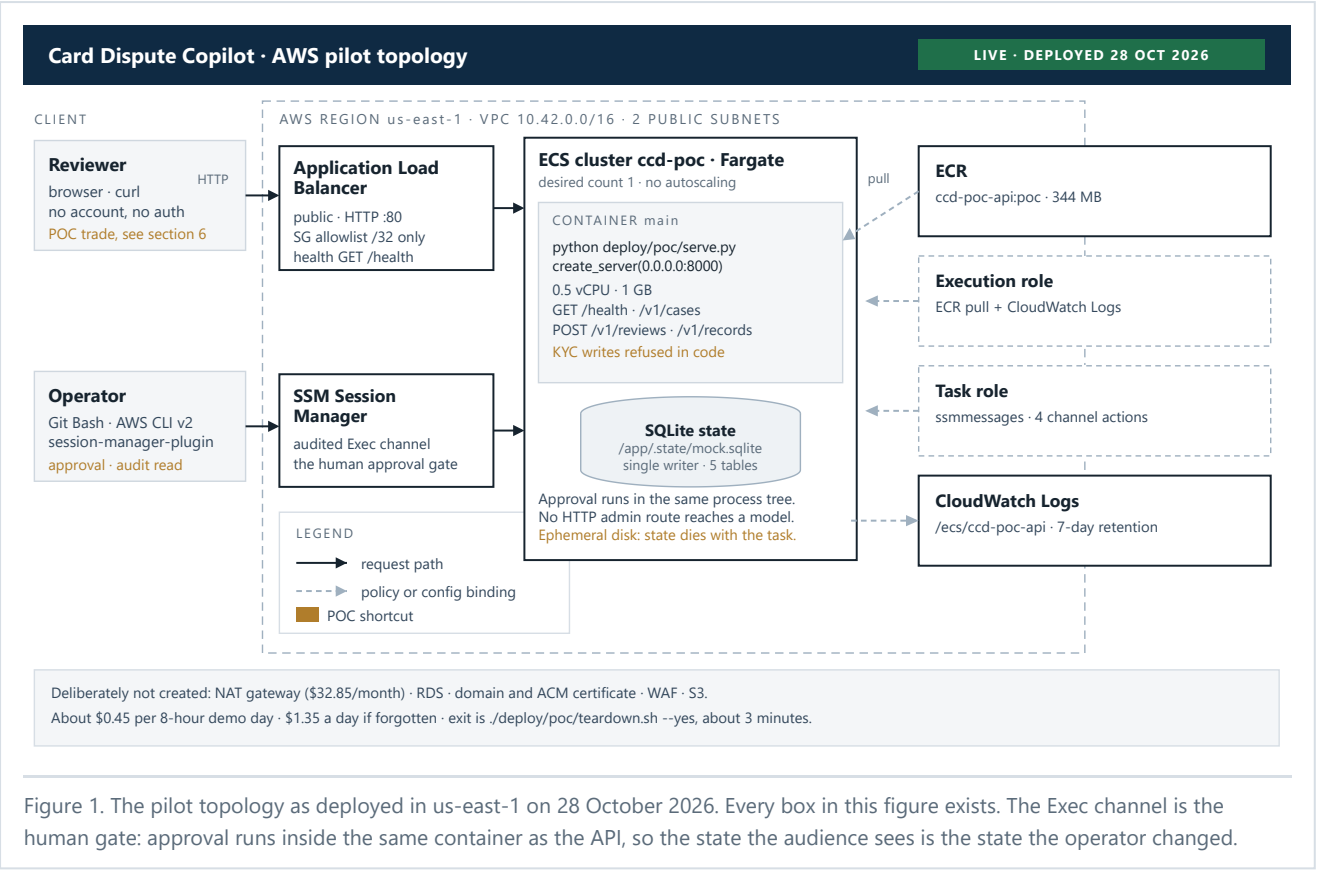


Figure 1. The pilot topology as deployed in us-east-1 on 28 October 2026. Every box in this figure exists. The Exec channel is the human gate: approval runs inside the same container as the API, so the state the audience sees is the state the operator changed.

4.1 Request flow

1. A reviewer or script calls the ALB DNS name over HTTP on port 80. The ALB security group accepts only one /32 address, taken once at deploy time.
2. The ALB forwards to target group ccd-poc-tg on port 8000. Only the ALB security group may reach port 8000 on the task.
3. The single Fargate task runs python deploy/poc/serve.py, a 20 line wrapper that calls the repository's own create_server with host 0.0.0.0. The repository CLI binds loopback only, which a load balancer can never reach. No application code changes.
4. State lives in one SQLite file on the task's ephemeral disk: reviews, records, keys, calls and audit.
5. The operator approves a review with aws ecs execute-command into the same task, running the repository's own CLI against the same file. Logs, including Exec session output, stream to CloudWatch.

4.2 Component choices and the constraint that forced each

Choice	Constraint that forced it
ECS Fargate, not Lambda	The application is a long-lived ThreadingHTTPServer with a mutable SQLite file. Lambda gives a fresh /tmp per sandbox and no stable single writer, so the review-then-record flow would break between invocations
One task, desired count 1	SQLite is single-writer, and autoscaling would fork the audit trail. p50 latency is 31 ms over 12 cases and the tail is one retry backoff, so a single task is ample for the pilot
Application Load Balancer, not API Gateway	The app speaks plain HTTP with no OpenAPI schema and no authorizer. API Gateway would add a proxy layer and an identity layer to a pilot that has no users
ECS Exec for the operator gate	Approval is deliberately CLI-only and reads the state file directly. Exec keeps that guarantee in the cloud without adding an HTTP admin route the model could reach
Ephemeral disk, not EFS	Approval and API are the same task, so no shared volume is needed. EFS adds mount targets, POSIX permissions and SQLite-over-NFS locking risk for a pilot that lasts one day
Public subnet, no NAT gateway	A NAT gateway is $32.85 a month and would dwarf every other line. A public IP gives egress for ECR and logs; ingress stays restricted to the ALB security group
HTTP for the demo window	ACM cannot issue for a bare ALB hostname. A one-day endpoint on fictional data, closed to one address, is a defensible POC trade. Section 6.4 prices the alternatives

4.3 Options considered and rejected

Option	Verdict	Why
ECS Fargate plus ALB	Chosen	Full control of health checks, Exec and logging at about $1.35 a day, every step scripted
ECS Express Mode	Fallback only	One call to a public HTTPS URL, but the container spec is single-container with no volumes and no enable-execute-command. That removes both the human gate and any state that survives a task replacement
AWS App Runner	Not available	Closed to new customers; AWS points new workloads at ECS Express Mode
Lambda plus Function URL	No	Needs a web adapter, /tmp-only SQLite, no single writer across invocations, no operator shell
EC2 with Docker	No	Cheaper per hour, but you own patching, TLS and the audit story, and it under-sells the architecture review
EKS	No	Control plane cost and operational surface for a single container

4.4 Status, stated plainly

The figure above is labelled LIVE because the topology is running: the ALB to task hop is healthy, and Exec against a real Fargate task is proven by the cloud run of every beat that needs it. What is proven, precisely:

- 26 Terraform-managed resources, applied, with a clean re-apply afterwards.
- The ALB forwards to one Fargate task on port 8000; the target is healthy and /health answers in 70 ms from the operator address.
- The scored run, executed inside the task, returns 12 of 12 with 0 format errors.
- The human gate is a real cloud action: an approval issued through ECS Exec changed service state, and a write without it is refused with 403.
- Every Exec session is captured in CloudWatch under /ecs/ccd-poc-api, stream prefix ecs-execute-command.

What is not proven, and should be said before anyone asks: availability beyond one task in one availability zone, behaviour under concurrent operators, and anything a bank would call production. Section 10 names the replacement for each shortcut.

5 Controls, and what a bank reviewer will still reject

5.1 What is true of this deployment

- **Fictional data only.** The dataset self-identifies as fictional_training_only, and the health endpoint says so. No card numbers, no cardholder names, no PII.
- **No credentials in the image.** No AWS keys, no model API keys. The live model path is refused because approved_by is empty.
- **Two least-privilege roles.** The execution role holds ECR pull and CloudWatch Logs. The task role holds the four ssmmessages channel actions that Exec needs, plus write access to one log group for session logging. No S3, no DynamoDB, no Secrets Manager.
- **The task is not internet-reachable except through the ALB.** Port 8000 accepts traffic only from the ALB security group. The public IP exists for egress.
- **Every Exec session is logged.** The break-glass channel is auditable in CloudWatch, including the operator approval. Verified 28 October: a session appears as its own stream under /ecs/ccd-poc-api, script-captured with command and exit code. This needs both the cluster-level configuration and the task-role Logs permission; with only the first, sessions run and nothing is recorded, which looks like success until someone asks for the log.
- **KYC read-only is enforced in code**, through dataset.allow_write and the 403 path. Moving to AWS did not weaken it.
- **Retention is 7 days** on the log group, so the pilot leaves no long-lived trace.

5.2 What a reviewer would still object to, with the answer for each

Objection	Pilot answer	Production answer
No TLS	Fictional data, one day, one allowed address, torn down after	ACM with 443 and TLS 1.2 minimum, or CloudFront in front of the ALB
No user authentication	There are no users; the write path requires an approved review	Cognito user pool or an IAM authorizer at the edge
No WAF	Single-address allowlist on the ALB security group	WAF managed rules at the edge
SQLite on ephemeral disk	One task, one writer, lifetime measured in hours	Aurora or DynamoDB behind a persistence adapter
Exec runs as root in the container	Documented AWS behaviour, session logged to CloudWatch	No Exec in the steady state; a signed admin API, Exec as break-glass only
Public ALB on a public IP	Dedicated VPC created for the pilot, address allowlist, deleted after the demo	Private subnets, private ALB, VPC endpoints
Data at rest	Fargate ephemeral storage is encrypted by default	KMS-managed volumes and database encryption
Bank system of record and write path in one process	Section 3.3 states it first: same task, same disk, same role, fictional data only, read-only enforced in code	Separate bank service; the copilot image holds no customer data and reaches the bank through a read-only PrivateLink contract

5.3 What this document is not

It is not a security attestation, a penetration test, a data protection impact assessment, or evidence of regulatory compliance. It is an engineering brief with named gaps. Anyone presenting it as more than that is overclaiming, and section 9 records that as a standing risk.

6 Cost

6.1 Price basis

Every rate comes from the public AWS Price List API for us-east-1, publication date 2026-10-28T00:46:31Z, read on 27 October 2026. Two lines are published but not price-list-exact: public IPv4 addresses and load balancer capacity units. Both are rounding errors at demo volume. Fargate, ALB hours, ECR storage and CloudWatch Logs are exact.

What was actually spent. The deployment ran for about ninety minutes on 28 October: one 0.5 vCPU task, one ALB, one public IPv4, ECR storage for one 83 MB image, and the Exec sessions on top. At the published rates that is about four cents. The forecast below is unchanged, and a demo day still costs forty-five cents.

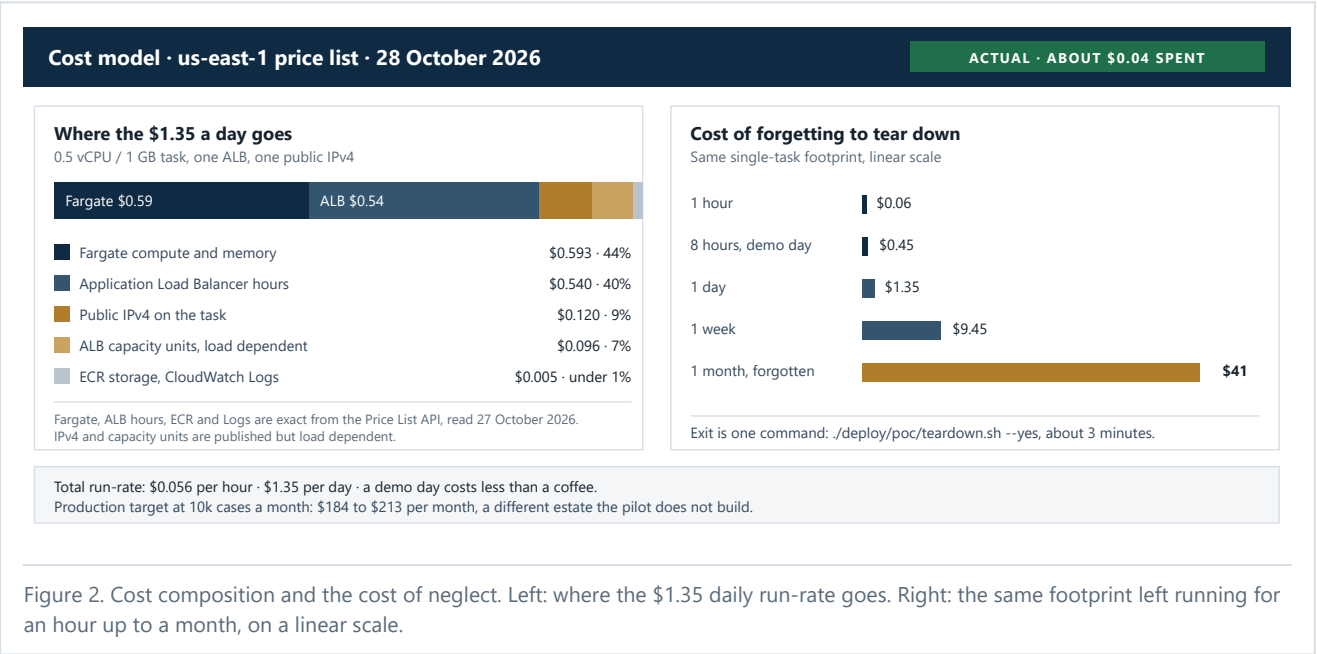


Figure 2. Cost composition and the cost of neglect. Left: where the $1.35 daily run-rate goes. Right: the same footprint left running for an hour up to a month, on a linear scale.

6.2 Scenarios

Scenario	Cost	Note
One demo day, 8 hours	$0.45	The intended use
Rehearsal plus demo, 2 days	$1.35	Dry run 29 Oct, defence 30 Oct
One week, left up	$9.45	Still cheap, but it should not happen
One month, forgotten	about $41	The failure case the teardown script exists to prevent
Larger task, 1 vCPU / 2 GB	about $1.18 more per day	Not needed: the workload is rules and I/O
Graviton (ARM64)	about 20 percent off compute, $0.15 a day	Available as a build flag; worth it only if there is spare time
Production target, 10k cases a month	$184 to $213 per month	Different estate: API Gateway, SQS, Step Functions, Lambda, DynamoDB, Bedrock, WAF

6.3 Guardrails

A $10 monthly budget with an email notification is the alarm. It sits well above the planned pilot spend of about $1.35 for two days and well below a forgotten month of $41, so it only fires when something is badly wrong. One limitation matters: **a budget action cannot stop ECS.** Budget actions reach IAM, SCPs and EC2 or RDS stop actions only. The kill switch for this pilot is the teardown script, not the budget.

Three CloudWatch alarms, about $0.10 per alarm-month each, would have caught every failure mode in section 8.4: ALB target 5xx count, healthy host count below 1, and ECS service CPU above 80 percent for five minutes.

6.4 What is deliberately not built, with the monthly cost avoided

Not built	Monthly cost avoided	Why it is not needed for a one-day pilot
NAT gateway	$32.85	A public IP gives the task egress without a NAT
RDS or Aurora	$12 to $15 at the smallest size	SQLite in one task is the correct shape for a pilot
WAF	$5 plus requests	The allowlist does the job for one address for one day
Route 53 zone and certificate	$0.50 plus a domain	No domain is owned or needed
CloudFront	pennies at this volume	Only if TLS is required on the pilot endpoint

7 Delivery plan

7.1 Effort, by stage

Stage	Effort	Mostly spent on
Prerequisites on the workstation	15 min	AWS CLI, Session Manager plugin, Docker running, identity check
Provision and deploy	25 to 35 min	ALB and target group activation, first task start
Build and push the image	10 to 15 min	First build only; the image is 342 MB
Verify and rehearse	20 min	The five checks in section 7.3, then the seven beats
Teardown	3 min	Ordered deletes, then a confirmation sweep

Measured on 28 October: 95 minutes from empty account to a healthy endpoint on a first attempt, including three script defects that had to be found and fixed, and two credential expiries. The second attempt is about 15 minutes because every phase is idempotent and the image is already in ECR.

7.2 Prerequisites

Item	Status, 28 Oct 2026, account 104005345381	How to verify
AWS account able to create ECR, ECS, ELB, IAM, CloudWatch and security groups	Present: admin role, org member o-u9p6hbqjuf, budget alarm at 1,000 USD per month	aws sts get-caller-identity
AWS CLI v2	Installed, 2.39.0, into .local/tools with no admin rights	aws --version
Session Manager plugin	Installed, 1.2.896.0, after one elevated click; required for the approval beat	session-manager-plugin --version
Terraform	1.17.0 local, AWS provider 6.70.0	terraform version
Docker Desktop	Installed, daemon reachable	docker info prints a server section
A VPC with two subnets	This account has no default VPC, so the stack creates 10.42.0.0/16 with two public subnets	terraform output vpc_id
Fargate vCPU quota	512 vCPU, ample for a 0.5 vCPU task	aws service-quotas get-service-quota, code L-3032A538
Region	us-east-1	terraform output region
TMPDIR	Must be a native Windows path or terraform init fails with Access is denied; phases.sh sets it	echo $TMPDIR

7.3 Verification before anyone watches

Run these in order. The last one is the check people skip, and it is the one that predicts whether the approval beat works.

1. curl the ALB hostname /health and expect fictional_training_only true.
2. aws elbv2 describe-target-health on ccd-poc-tg and expect healthy.
3. aws ecs describe-tasks and expect the container managed agent ExecuteCommandAgent to be RUNNING.
4. Run the scored run against the cloud URL and expect the same 12 of 12 with 0 format errors.

5. Read the audit table through Exec and expect an ordered trail ending in the last action performed.

7.4 Who does what

Role	Person or group	Accountable for
Pilot operator	Ken (FDE)	Provisioning, verification, demo execution, teardown
Delivery lead	Coforge pod lead	Spend approval, audience management, sign-off on claims
Platform owner	Coforge AWS account administrator	Account, guardrails, credential issuance, quota
Bank reviewer	Lumen platform and model risk	Objection register, production criteria, model sign-off

8 Operating the pilot

8.1 Seven beats, each one a claim on screen

Beat	Command	What the audience sees	What it proves
1 Boundary is real	health, cases	200 from an amazonaws.com hostname, 12 cases	The service is public and governed
2 Score is unchanged	score	12 of 12, 0 format errors, p50 17 ms, p95 3.96 s, $0.00 per case	Moving to cloud did not launder correctness
3 A draft arrives	request CARD-D01, pending	201 with rev id and draft hash, review in queue	The queue is real state, not a fixture
4 Human gate	approve rev, pending	approved, then an empty queue	Approval is a deliberate human act
5 Creation and idempotency	record rev	201 created, 200 replay, 409 conflict, 403 tampered approval	Writes are gated and replay-safe
6 Duplicate protection	CARD-D02 then CARD-D09, different idempotency keys	second call 409 duplicate_case with existing record id	The logical key is enforced. Reusing the same idempotency key instead returns 409 idempotency_conflict, which proves the other guard
7 Audit trail	audit	ordered rows: tool_read, review_requested, human_approved, record_created, write_replayed, idempotency_conflict, unapproved_write_attempt, duplicate_prevented	Every step is attributable and hash-bound

Warm up 10 minutes before the audience. Capture the audit table to a file outside the task before starting, and do not redeploy between warm-up and demo: a new task means a new empty state file, and beats 3 to 6 would start from nothing.

Leave a second between the approve beat and the record beat. Approval writes and the API reads share one SQLite file, and a read that lands inside the write came back 404 once in testing; a one-second gap removed it, and the same gap is worth keeping on stage. Beat 6 needs a different idempotency key from beat 5, otherwise the second call fails on the key rather than on the logical record and you see 409 idempotency_conflict instead of 409 duplicate_case.

8.2 Fallback ladder

Level	Trigger	Action	Time to switch
1 Cloud	Normal	Run the beats against the ALB	0
2 Local container	ALB or task unhealthy and not fixed in 5 minutes	Same image, docker run on the laptop, same seven beats	2 min
3 Recorded evidence	No working endpoint at all	Appendix A outputs plus the audit file captured at warm-up	0

The ladder is stated out loud to the audience, not hidden. A team that can name its fallback is more credible than one that improvises.

8.3 Monitoring during the window

Signal	Where	What it tells you
Target health	describe-target-health	Whether the ALB can reach the task, the single most common failure
5xx count on the ALB	CloudWatch, per target group	Application errors versus infrastructure errors
Service CPU and memory	CloudWatch, ECS metrics	A runaway loop or a memory ceiling
Log tail	aws logs tail /ecs/ccd-poc-api	The actual traceback, in one command
Exec agent status	describe-tasks managed agents	Whether the human gate will work at all

8.4 Failure modes and the fix

Symptom	Cause	Fix
CannotPullContainerError	Execution role missing the managed policy, or architecture mismatch	Attach AmazonECSTaskExecutionRolePolicy; build for linux/amd64 for X86_64
Target unhealthy	Wrong health path or port, or the app bound to loopback	Health check must be /health on 8000; the wrapper must bind 0.0.0.0
ALB returns 503	No healthy targets	describe-target-health, then the log group
Exec fails with TargetNotConnected	Service created without enable-execute-command, or task role lacks ssmmessages	Re-run update-service with the flag; check the task role policy
Writes fail with readonly database	An Exec session running as root left a root-owned journal file	chown the state directory inside the task
Exec works but the URL 404s	Tool names are per case in the dataset, not global	Read GET /v1/cases, then request a tool the case actually has

8.5 Teardown, in order, with confirmation

1. Scale the service to zero, then delete it, so Fargate billing stops first.
2. Delete the cluster.
3. Delete the load balancer and wait until it is gone before deleting listeners or target groups, or the call fails with ResourceInUse.
4. Delete the target group and both security groups.
5. Delete the log group; delete the ECR repository with force.

6. Delete the inline policy, then both IAM roles.
7. Confirm with five list and describe calls that nothing billing remains.

   If CloudFront was added for TLS, disable the distribution, wait for status Deployed, then delete it. An enabled distribution cannot be deleted.

9 Risk register

Risk	Likelihood	Impact	Mitigation	Owner
First-time deployment overruns the 90 minute estimate	Medium	Medium	Run the idempotent script once in a sandbox account before the demo date; rehearse the fallback	Operator
Fargate vCPU quota exhausted on a busy account	Low	High	Check quota L-3032A538 before provisioning; the task needs 0.5 vCPU	Platform owner
Address allowlist locks the team out after an IP change	Medium	Medium	Take the CIDR from one place, keep a console path, re-authorize the group in one call	Operator
Task replacement wipes SQLite state mid-demo	Low	High	No redeploy after warm-up; audit baseline captured off-task; redo sequence rehearsed	Operator
Reading the pilot as production-ready	Medium	High	Section 4.4 lists what is proven and what is not; section 10 replaces each shortcut by name	Delivery lead
Short-lived credentials expire inside the demo window	Medium	High	A long-lived IAM user key for the window, or a fresh console export immediately before; the ExpiredToken path is in the runbook	Operator
Credential exposure during setup	Low	High	SSO or short-lived credentials, no keys in the image, CI rejects committed env files	Platform owner
Cost drift if the stack is left running	Low	Low	Budget alarm, teardown script, scale to zero between sessions	Operator
Scope creep into live model calls	Medium	Medium	Model stays replay-only until a qualification record and a bank signature exist	Delivery lead
Reviewer reads the shared process as custody of customer data	Medium	High	Section 3.3 raises it before they do, and 10.1 names the replacement; the pilot claims a contract boundary, not custody	Delivery lead
Demo day network failure	Medium	Medium	Fallback ladder in section 8.2, rehearsed at level 2	Operator

9.1 The two risks that actually decide whether this works

Every row above is manageable. Two of them decide the outcome, and both are behavioural rather than technical.

- **Overclaiming.** If anyone describes the pilot as deployed before it is, the first question from a bank reviewer becomes "which account, which controls", and the answer is a design document. The mitigation is cheap and absolute: the words "pilot ready, not yet provisioned" until the seven beats run against an amazonaws.com hostname.
- **First-time deploy overrun.** The 90 minute estimate assumes someone has done it before. If the demo date is fixed, run deploy.sh once in a throwaway window, delete everything, then run it again on the day. The second run is about 25 minutes because the script is idempotent.

  Neither risk costs money to close. Both cost credibility if left open.

10 From pilot to production

10.1 The three shortcuts, each with a named replacement

Pilot choice	Production replacement	Why it cannot carry forward
One task in one AZ	Two or more tasks across AZs behind the ALB	Availability, and it forces the state question into the open
SQLite on ephemeral disk	Aurora PostgreSQL or DynamoDB behind the existing store interface	Durability, concurrency, backup, restore
No authentication on the API	Cognito user pool or an IAM-authenticated API Gateway	Operator identity has to be provable, not asserted
HTTP on the ALB	ACM with 443, TLS 1.2 minimum, or CloudFront	A bank requirement, not a preference
Address allowlist	WAF managed rules plus private subnets	The allowlist is a convenience, not a control
Exec for approval	A signed admin API, Exec as break-glass only	Approval must be a first-class, audited action
Replay-only model	Live model behind the qualification gate	Already designed; needs the adapter and a signature
Bank mock and dispute platform in one process (section 3.3)	A separate bank service, read-only contract, copilot image carrying no customer data	Custody: the component that writes decisions must not hold the system of record

10.2 Phases and entry criteria

Phase	Scope	Effort estimate	Exit criterion
P0 Pilot, this document	One container, public boundary, human gate, audit	1 day	Seven beats demonstrated, teardown confirmed
P1 Hardening	TLS, authentication, private subnets, persistence adapter, CI deploy on main	2 to 3 weeks for one engineer	A bank reviewer can walk the controls matrix without exceptions
P2 Model live	Bedrock adapter, model access, qualification run at the registered schema hash, guardrail decision, sign-off	3 to 4 weeks	A named bank approver in models.toml, and a qualification record that passes

10.3 If the model is ever turned on, it is Bedrock

- There is no Bedrock adapter today. src/model_clients.py allows only azure_openai and anthropic_foundry, and model_from_env raises for anything else.
- The gate needs two things the pilot does not have: a passed qualification record at the registered schema hash, and a non-empty approved_by in models.toml.
- Structured output uses outputConfig.textFormat.type as json_schema, and the schema field is a JSON string, not an object. The repository's cue schemas already satisfy the supported subset.
- First use compiles a grammar, which can take up to about 90 seconds, so warm the model before any demo and treat a timeout as retryable.
- Guardrails cost roughly nine times the tokens of the call itself. At pilot volume the model is about $0.005 a day, so cost is not the reason to defer it. Signature and evidence are.

11 Decisions and status

#	Decision	Status, 28 October 2026	What remains
D1	Spend and account	**Granted.** Deployed to account 104005345381, us-east-1; about four cents spent	Keep the residual cap and the teardown script as the only standing conditions
D2	Transport security for the demo	**Decided: HTTP** with a one-address allowlist	CloudFront with TLS if a bank stakeholder joins the session
D3	Framing	**Adopted:** "identical score, different boundary", stated first	Section 4.4 is the one paragraph to keep repeating
D4	Production hardening review	**Open.** Needs a date and an owner for P1	Two to three weeks of engineering, pod lead plus bank platform

Two notes that still apply. The account must be a sandbox or a dedicated pilot account, not a shared production account, because the pilot creates and deletes IAM roles and a load balancer; this one is a Coforge member account with a 1,000 USD monthly budget, so a four cent pilot is invisible to the alarm and the teardown script, not the budget, is the kill switch. And the demo endpoint must be torn down or scaled to zero after the defence window, because the load balancer bills whether or not anyone is watching.

12 Assumptions and limits

- Prices are us-east-1 list prices on 28 October 2026, without reservations, savings plans, support plan or tax. They will be wrong in another region and after the next price change.
- Effort estimates assume one engineer who knows this repository. A first-time AWS operator should add time for the prerequisites, not for the design.
- The pilot processes 12 fictional dev cases. Throughput claims do not extrapolate from 12 cases, and none are made here.
- The bank in this pilot is a mock that shares the platform's process (section 3.3). Nothing in this document is evidence about a real bank integration, its latency, or its data contract.
- The scored run uses the repository's determinism and replay recordings; it makes no model calls, so latency here says nothing about model latency.
- Nothing in this document is a security attestation, a compliance statement or legal advice.
- Distribution is Internal. Sending it to a bank requires the review that the classification implies.

Appendix A, verification evidence, 28 October 2026

Everything below was executed against the deployed pilot in account 104005345381, us-east-1, on 28 October 2026. Beats that must read or write the state file run inside the single task through ECS Exec, which is the point: the operator acts on cloud state, not on a copy.

Check	Result
terraform apply, six phases	26 resources, exit 0; a later re-apply reports no changes except the two intended ones
Deployment path	registry, core, image, edge, service, verify, through phases.sh
Target health	10.42.0.206 healthy, from the ALB target group
GET /health through the ALB	200 in 70 ms, status ok, fictional_training_only true
GET /v1/cases	200, 12 cases, each with case_id, intake_source and documents
Scored run inside the task	12 of 12 passed, 0 format errors, p50 17 ms, p95 3,957 ms, $0.00 per case
ExecuteCommandAgent	RUNNING on the running task
POST /v1/reviews	201, rev_3af0e66ae161, draft hash 8b0c0110aa7d617171f68039688aaeaf2b08b032117cde3055bb7ca403fa5665
pending, through Exec	one review, status pending, reviewer null, draft and hash visible
approve rev_3af0e66ae161 --reviewer ken	approved, reviewer recorded as ken
record path	201 created dispute-d01, 200 replay identical, a changed draft refused because the hash differs, 200 readback
Duplicate guard	CARD-D02 approved and created dispute-double-02 from rev_854d00942293; CARD-D09 from rev_d1dc3cc4b0b5, same logical key, different idempotency key, returned 409 duplicate_case with existing_record_id dispute-double-02
Audit table	278 ordered rows, ending in duplicate_prevented for CARD-D09
Exec session log	stream ecs-execute-command-a8faq7tqo6buuzkuc7kcdva6ja under /ecs/ccd-poc-api, script-captured with COMMAND and COMMAND_EXIT_CODE
Spend at the time of writing	about four cents

Three defects were found during the deployment and fixed in the repository, not in the console: the core phase did not target the VPC network resources, so the first apply created a VPC with no subnets; the image build passed deploy/ as its context instead of the repository root; and the Exec logging step called update-service with a parameter this API version does not have, which is why logging now goes through update-cluster in phase 5b. The load balancer left behind by an interrupted apply was imported into state by ARN rather than recreated, and the Exec logging claim in section 5.1 was only true after the task role was given write access to the log group.

Appendix B, command reference

Prerequisites, once, in PowerShell as administrator:

```
winget install --id Amazon.AWSCLI -e
# Session Manager plugin: MSI from the AWS documentation page
aws configure          # or aws configure sso, region us-east-1, output json
session-manager-plugin
docker info            # must print a server section
```

The same stack is also defined in Terraform, in deploy/poc/terraform, with up.sh and down.sh wrappers. The script below is the imperative path and creates identical resource names, so the seven beats work against either.

Deploy, from Git Bash in the repository root:

```
export AWS_REGION=us-east-1
export ALLOW_CIDR="$(curl -s https://checkip.amazonaws.com)/32"
./deploy/poc/deploy.sh
./deploy/poc/demo.sh health
./deploy/poc/demo.sh score
```

Verify the human gate before the audience:

```
aws ecs describe-tasks --cluster ccd-poc --tasks "$(aws ecs list-tasks \
  --cluster ccd-poc --service-name ccd-poc-api --query 'taskArns[0]' --output text)" \
  --query 'tasks[0].containers[0].managedAgents[0].lastStatus' --output text
```

Run the seven beats, then tear down:

```
./deploy/poc/demo.sh request CARD-D01
./deploy/poc/demo.sh pending
./deploy/poc/demo.sh approve rev_xxx --reviewer ken
./deploy/poc/demo.sh record rev_xxx
./deploy/poc/demo.sh audit
./deploy/poc/teardown.sh --yes
```

Appendix C, source register

All pages retrieved 27 and 28 October 2026. Prices from the AWS Price List API, which is the machine-readable source behind the pricing pages.

Topic	Reference
ECS Exec prerequisites and limits	docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-exec.html
Session Manager plugin installation	docs.aws.amazon.com/systems-manager/latest/userguide/session-manager-working-with-install-plugin.html
ECS service load balancing and health checks	docs.aws.amazon.com/AmazonECS/latest/developerguide/service-load-balancing.html
Fargate task sizes and pricing	aws.amazon.com/fargate/pricing
Application Load Balancer pricing	docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html
ECR pricing	aws.amazon.com/ecr/pricing
CloudWatch Logs retention	docs.aws.amazon.com/AmazonCloudWatch/latest/logs/Working-with-log-groups-and-log-streams.html
Budgets and budget actions	docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html
AWS Price List API, us-east-1	pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonECS/current/us-east-1/index.json
App Runner availability change	docs.aws.amazon.com/apprunner/latest/dg/apprunner-availability-change.html
ECS Express Mode overview	docs.aws.amazon.com/AmazonECS/latest/developerguide/express-service.html
Bedrock structured outputs	docs.aws.amazon.com/bedrock/latest/userguide/structured-output.html
SQLite over network filesystems	sqlite.org/useovernet.html